



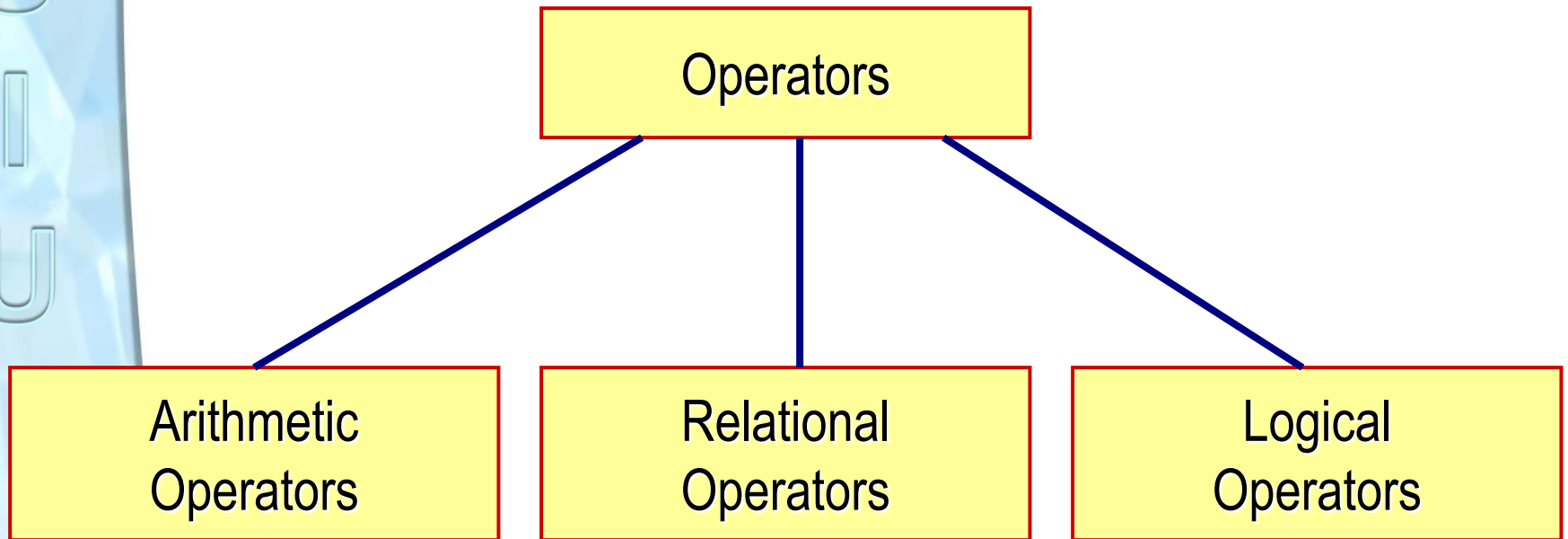
United International University
QUEST FOR EXCELLENCE



CSI 121: Structured Programming Language Use of Operators

Salekul Islam
United International University

Operators in Expressions



Working with arithmetic expressions

- Basic arithmetic operators: $+$, $-$, $*$, $/$, $\%$
- **Precedence**: one operator can have a higher priority, or *precedence*, over another operator.
 - Example: $*$ has a higher precedence than $+$
 - $a + b * c$
 - if necessary, you can always use parentheses in an expression to force the terms to be evaluated in any desired order.
- **Associativity**: Expressions containing operators of the same precedence are evaluated either from left to right or from right to left, depending on the operator. This is known as the *associative* property of an operator
 - Example: $+$ has a *left to right* associativity
- In C there are many more operators -> later in this course !

Working with arithmetic expressions

```
#include <stdio.h>
int main (void)
{
    int a = 100;
    int b = 2;
    int c = 25;
    int d = 4;
    int result;
    result = a - b; // subtraction
    printf ("a - b = %i\n", result);
    result = b * c; // multiplication
    printf ("b * c = %i\n", result);
    result = a / c; // division
    printf ("a / c = %i\n", result);
    result = a + b * c; // precedence
    printf ("a + b * c = %i\n", result);
    printf ("a * b + c * d = %i\n", a * b + c * d);
    return 0;
}
```

Integer arithmetic and the unary minus operator

```
// More arithmetic expressions
#include <stdio.h>
int main (void)
{
    int a = 25;
    int b = 2;
    float c = 25.0;
    float d = 2.0;
    printf ("6 + a / 5 * b = %i\n", 6 + a / 5 * b);
    printf ("a / b * b = %i\n", a / b * b);
    printf ("c / d * d = %f\n", c / d * d);
    printf ("-a = %i\n", -a);
    return 0;
}
```

The modulus operator

```
// The modulus operator
#include <stdio.h>
int main (void)
{
    int a = 25, b = 5, c = 10, d = 7;
    printf ("a %% b = %i\n", a % b);
    printf ("a %% c = %i\n", a % c);
    printf ("a %% d = %i\n", a % d);
    printf ("a / d * d + a %% d = %i\n",
            a / d * d + a % d);

    return 0;
}
```

Modulus operator: %

Binary operator

Gets the remainder resulting from integer division

% has equal precedence to * and /

Integer and Floating-Point Conversions

- Assign an integer value to a floating variable: does not cause any change in the value of the number; the value is simply converted by the system and stored in the floating
- Assign a floating-point value to an integer variable: the decimal portion of the number gets truncated.
- Integer arithmetic (division):
 - int divided to int => result is integer division
 - int divided to float or float divided to int => result is real division (floating-point)

Integer and Floating-Point Conversions

```
// Basic conversions in C
#include <stdio.h>
int main (void)
{
    float f1 = 123.125, f2;
    int i1, i2 = -150;
    char c = 'a';
    i1 = f1; // floating to integer conversion
    printf ("%f assigned to an int produces %i\n", f1, i1);
    f1 = i2; // integer to floating conversion
    printf ("%i assigned to a float produces %f\n", i2, f1);
    f1 = i2 / 100; // integer divided by integer
    printf ("%i divided by 100 produces %f\n", i2, f1);
    f2 = i2 / 100.0; // integer divided by a float
    printf ("%i divided by 100.0 produces %f\n", i2, f2);
    f2 = (float) i2 / 100; // type cast operator
    printf ("(float) %i divided by 100 produces %f\n", i2, f2);
    return 0;
}
```


The Type Cast Operator

- `f2 = (float) i2 / 100; // type cast operator`
- The type cast operator has the effect of converting the value of the variable `i2` to type `float` for purposes of evaluation of the expression.
- This operator does NOT permanently affect the value of the variable `i2`;
- **The type cast operator has a higher precedence than all the arithmetic operators except the unary minus and unary plus.**
- Examples of the use of the type cast operator:
 - `(int) 29.55 + (int) 21.99` results in `29 + 21`
 - `(float) 6 / (float) 4` results in `1.5`
 - `(float) 6 / 4` results in `1.5`

The assignment operators

- The C language permits you to join the arithmetic operators with the assignment operator using the following general format: `op=`, where `op` is an arithmetic operator, including `+`, `-`, `*`, `/`, and `%`.
- `op` can also be a logical or bit operator => later in this course
- Example:

```
count += 10;
```

 - Equivalent with:

```
count=count+10;
```
- Example: precedence of `op=`:

```
a /= b + c
```

 - Equivalent with:

```
a = a / (b + c)
```
 - addition is performed first because the addition operator has higher precedence than the assignment operator

Increment operator

- Because addition by 1 is a very common operation in programs, a special operator was created in C for this.
- *Increment operator*: the expression $++n$ is equivalent to the expression $n = n + 1$.
- *Decrement operator*: the expression $--n$ is equivalent to the expression $n = n - 1$
- Increment and decrement operators can be placed in front (*prefix*) or after (*postfix*) their operand.
- The *difference between prefix and postfix*:
- Example: if $n=4$:
 - $a = n++$ leads to $a=4, n=5$ (L-value $\neq$ R-value)
 - $a = ++n$ leads to $a=5, n=5$ (L-value = R-value)



Suppose, if you buy two noodles, you will get 1 free. The price of each noodle is 15 tk. If you want take 53 noodles home, how much do you have to pay?

Relational Operators

Operator	Meaning
==	Is equal to
!=	Is not equal to
<	Is less than
<=	Is less or equal
>	Is greater than
>=	Is greater or equal

The relational operators have lower precedence than all arithmetic operators:
 $a < b + c$ is evaluated as $a < (b + c)$

ATTENTION ! Do not confuse:
the “is equal to” operator == and the “assignment” operator =

ATTENTION when comparing floating-point values !
Only < and > comparisons make sense !

Examples

$10 > 20$ is false

$25 < 35.5$ is true

$12 > (7 + 5)$ is false

- When arithmetic expressions are used on either side of a relational operator, the arithmetic expressions will be evaluated first and then the results compared.

$a + b > c - d$ is the same as $(a+b) > (c+d)$

Examples

- Sample code segment in C

```
if (x > y)
    printf ("%d is larger\n", x);
else
    printf ("%d is larger\n", y);
```

Logical Operators

- There are two logical operators in C (also called logical connectives).
 - && → Logical AND
 - || → Logical OR
- What they do?
 - They act upon operands that are themselves logical expressions.
 - The individual logical expressions get combined into more complex conditions that are true or false.

Logical Operators

- Logical AND
 - Result is true if both the operands are true.
- Logical OR
 - Result is true if at least one of the operands are true.

X	Y	X && Y	X Y
FALSE	FALSE	FALSE	FALSE
FALSE	TRUE	FALSE	TRUE
TRUE	FALSE	FALSE	TRUE
TRUE	TRUE	TRUE	TRUE